RA Image Augmentation and Model Training

This section uses the provided Rheumatoid Arthritis image to generate a dataset and train the model.

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import transforms, models
from PIL import Image
import os

# Initialize data storage
aug_data = []
# FIX: Use the path to the file uploaded to Colab, not your local C: drive
image_path = '/content/RA Image.jfif'

try:
    # Load the local image
    img = Image.open(image_path).convert('RGB')

    # Create output directory
    os.makedirs('augmented_images', exist_ok=True)
    img.save('augmented_images/original.jpg')

    # Define Augmentation Pipeline
    augmentation_transform = transforms.Compose([
        transforms.RandomRotation(30),
        transforms.RandomHorizontalFlip(),
        transforms.RandomVerticalFlip(),
        transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),
        transforms.ColorJitter(brightness=0.2, contrast=0.2),
        transforms.ToTensor(),
    ])

    # Generate 40 variations
    print("Generating 40 augmented images...")
    for i in range(40):
        aug_img_tensor = augmentation_transform(img)
        aug_data.append(aug_img_tensor)
        # Save for manual inspection
        transforms.ToPILImage()(aug_img_tensor).save(f'augmented_images/aug_{i}.jpg')

    print(f"Successfully saved 40 images to 'augmented_images/' folder.")

except Exception as e:
    print(f"Error processing local image: {e}")
```

```
Generating 40 augmented images...
Successfully saved 40 images to 'augmented_images/' folder.
```

```
# Training Loop using ResNet18
if 'aug_data' in locals() and len(aug_data) > 0:
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"Training on: {device}")

    # Load pre-trained ResNet18
```

```
    model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
    # Binary classification head
    model.fc = nn.Linear(model.fc.in_features, 2)
    model = model.to(device)

    # Data preparation
    inputs = torch.stack(aug_data).to(device)
    labels = torch.zeros(len(aug_data), dtype=torch.long).to(device)

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)

    model.train()
    for epoch in range(5):
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        print(f"Epoch {epoch+1}/5, Loss: {loss.item():.4f}")
    print("Training process finished successfully.")
else:
    print("Augmentation failed; cannot start training.")
```

```
Training on: cpu
Epoch 1/5, Loss: 0.6182
Epoch 2/5, Loss: 1.2113
Epoch 3/5, Loss: 0.9065
Epoch 4/5, Loss: 0.5068
Epoch 5/5, Loss: 0.1414
Training process finished successfully.
```

Model Evaluation

In this section, we split the augmented data into training and validation sets to evaluate the model's performance on unseen variations.

```
from torch.utils.data import DataLoader, TensorDataset, random_split

if 'aug_data' in locals() and len(aug_data) > 0:
    # 1. Prepare Dataset and Split
    full_dataset = TensorDataset(
        torch.stack(aug_data),
        torch.zeros(len(aug_data), dtype=torch.long)  # Note: These are placeholder labels (all 0)
    )

    train_size = int(0.8 * len(full_dataset))
    val_size = len(full_dataset) - train_size
    train_dataset, val_dataset = random_split(full_dataset, [train_size, val_size])

    train_loader = DataLoader(train_dataset, batch_size=8, shuffle=True)
    val_loader = DataLoader(val_dataset, batch_size=8, shuffle=False)

    # 2. Re-initialize model
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
    model.fc = nn.Linear(model.fc.in_features, 2)
    model = model.to(device)
```

```
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)

    # 3. Training and Validation Loop
    epochs = 5
    for epoch in range(epochs):
        model.train()
        train_loss = 0
        for inputs, labels in train_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
            train_loss += loss.item()

        # Evaluation Phase
        model.eval()
        val_loss = 0
        correct = 0
        total = 0
        with torch.no_grad():
            for inputs, labels in val_loader:
                inputs, labels = inputs.to(device), labels.to(device)
                outputs = model(inputs)
                loss = criterion(outputs, labels)
                val_loss += loss.item()
                _, predicted = torch.max(outputs.data, 1)
                total += labels.size(0)
                correct += (predicted == labels).sum().item()

        avg_train_loss = train_loss / len(train_loader)
        avg_val_loss = val_loss / len(val_loader)
        val_acc = 100 * correct / total

        print(f'Epoch {epoch+1}: Train Loss: {avg_train_loss:.4f} | Val Loss: {avg_val_loss:.4f} | Val Acc: {val_acc:.2f}%')

else:
    print("No data available for evaluation.")
```

```
Epoch 1: Train Loss: 0.6200 | Val Loss: 0.3584 | Val Acc: 87.50%
Epoch 2: Train Loss: 0.0401 | Val Loss: 0.0000 | Val Acc: 100.00%
Epoch 3: Train Loss: 0.0036 | Val Loss: 0.0000 | Val Acc: 100.00%
Epoch 4: Train Loss: 0.0012 | Val Loss: 0.0000 | Val Acc: 100.00%
Epoch 5: Train Loss: 0.0004 | Val Loss: 0.0000 | Val Acc: 100.00%
```

Binary Classification: Normal vs RA

This section handles multiple classes, generates augmented datasets for both, and trains the model with real labels.

```
import os
import torch
from torchvision import transforms
from PIL import Image

# Paths to your representative images
class_paths = {
```

```
    0: '/content/Normal Image.jfif', # Update if your filename is different
    1: '/content/RA Image.jfif'
}

# Augmentation Pipeline
augmentation_transform = transforms.Compose([
    transforms.RandomRotation(30),
    transforms.RandomHorizontalFlip(),
    transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),
    transforms.ColorJitter(0.2, 0.2),
    transforms.ToTensor(),
])

all_tensors = []
all_labels = []

for label, path in class_paths.items():
    if os.path.exists(path):
        img = Image.open(path).convert('RGB')
        print(f"Generating data for class {label}...")
        for _ in range(40):
            all_tensors.append(augmentation_transform(img))
            all_labels.append(label)
    else:
        print(f"Warning: Path {path} not found. Please upload the file.")

if len(all_tensors) > 0:
    X = torch.stack(all_tensors)
    y = torch.tensor(all_labels)
    print(f"Dataset ready: {X.shape[0]} images across {len(set(all_labels))} classes.")
```

```
Warning: Path /content/Normal Image.jfif not found. Please upload the file.
Generating data for class 1...
Dataset ready: 40 images across 1 classes.
```

```
from torch.utils.data import DataLoader, TensorDataset, random_split

if 'X' in locals():
    dataset = TensorDataset(X, y)
    train_size = int(0.8 * len(dataset))
    val_size = len(dataset) - train_size
    train_db, val_db = random_split(dataset, [train_size, val_size])

    train_loader = DataLoader(train_db, batch_size=8, shuffle=True)
    val_loader = DataLoader(val_db, batch_size=8, shuffle=False)

    # Model Setup
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
    model.fc = nn.Linear(model.fc.in_features, 2)
    model = model.to(device)

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)

    # Training Loop
    for epoch in range(10):
        model.train()
        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)
```

```
            optimizer.zero_grad()
            loss = criterion(model(images), labels)
            loss.backward()
            optimizer.step()

        # Simple Validation
        model.eval()
        correct = 0
        with torch.no_grad():
            for images, labels in val_loader:
                images, labels = images.to(device), labels.to(device)
                preds = model(images).argmax(dim=1)
                correct += (preds == labels).sum().item()

        print(f"Epoch {epoch+1} | Val Acc: {100 * correct / val_size:.2f}%")
```

```
Epoch 1 | Val Acc: 0.00%
Epoch 2 | Val Acc: 0.00%
Epoch 3 | Val Acc: 25.00%
Epoch 4 | Val Acc: 100.00%
Epoch 5 | Val Acc: 100.00%
Epoch 6 | Val Acc: 100.00%
Epoch 7 | Val Acc: 100.00%
Epoch 8 | Val Acc: 100.00%
Epoch 9 | Val Acc: 100.00%
Epoch 10 | Val Acc: 100.00%
```

Verify 'Normal' Image Upload

Run this cell after uploading `Normal Image.jfif` to your `/content/` folder.

```
import matplotlib.pyplot as plt
from PIL import Image
import os

normal_path = '/content/Normal Image.jfif'

if os.path.exists(normal_path):
    img = Image.open(normal_path)
    plt.imshow(img)
    plt.title("Successfully Loaded: Normal Image")
    plt.axis('off')
    plt.show()
    print("You can now re-run the 'Binary Classification' cells above to train with both classes.")
else:
    print(f"File not found at {normal_path}. Please upload the file and ensure the name matches exactly.")
```

```
File not found at /content/Normal Image.jfif. Please upload the file and ensure the name matches exactly.
```

```
import os

# Updated to match the actual file name in /content/
file_name = 'Normal hand joint.jfif'
path = f'/content/{file_name}'

if os.path.exists(path):
    print(f'✅ Found: {path}')
else:
    print(f'❌ Not Found: {path}')
```

```
    print('\nAvailable files in /content/:')
    print(os.listdir('/content/'))
```

```
✅ Found: /content/Normal hand joint.jfif
```

```
import os
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import transforms, models
from PIL import Image
from torch.utils.data import DataLoader, TensorDataset, random_split

# 1. Define Paths and Transforms - UPDATED FILENAME
class_paths = {0: '/content/Normal hand joint.jfif', 1: '/content/RA Image.jfif'}
augmentation_transform = transforms.Compose([
    transforms.RandomRotation(30),
    transforms.RandomHorizontalFlip(),
    transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),
    transforms.ColorJitter(0.2, 0.2),
    transforms.ToTensor(),
])

# 2. Generate Balanced Dataset
all_tensors, all_labels = [], []
for label, path in class_paths.items():
    if os.path.exists(path):
        img = Image.open(path).convert('RGB')
        print(f"Augmenting class {label}...")
        for _ in range(40):
            all_tensors.append(augmentation_transform(img))
            all_labels.append(label)
    else:
        print(f"Error: Missing file {path}")

if len(set(all_labels)) < 2:
    print("Stop: Both classes are required for binary classification.")
else:
    X = torch.stack(all_tensors)
    y = torch.tensor(all_labels)
    dataset = TensorDataset(X, y)

    # 3. Split Data
    train_size = int(0.8 * len(dataset))
    val_size = len(dataset) - train_size
    train_db, val_db = random_split(dataset, [train_size, val_size])
    train_loader = DataLoader(train_db, batch_size=8, shuffle=True)
    val_loader = DataLoader(val_db, batch_size=8, shuffle=False)

    # 4. Initialize Model
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
    model.fc = nn.Linear(model.fc.in_features, 2)
    model = model.to(device)

    criterion = nn.CrossEntropyLoss()
    # Added weight_decay for better generalization
    optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-4)

    # 5. Training Loop
```

```python
# 5. Training Loop
print(f"Training on {device}...")
for epoch in range(10):
    model.train()
    total_loss = 0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

    model.eval()
    correct = 0
    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            preds = model(images).argmax(dim=1)
            correct += (preds == labels).sum().item()

    print(f"Epoch {epoch+1}/10 | Loss: {total_loss/len(train_loader):.4f} | Val Acc: {100 * correct / val_size:.2f}%")
```

```
Augmenting class 0...
Augmenting class 1...
Training on cpu...
Epoch 1/10 | Loss: 0.4823 | Val Acc: 100.00%
Epoch 2/10 | Loss: 0.1192 | Val Acc: 81.25%
Epoch 3/10 | Loss: 0.0202 | Val Acc: 93.75%
Epoch 4/10 | Loss: 0.0020 | Val Acc: 100.00%
Epoch 5/10 | Loss: 0.3923 | Val Acc: 100.00%
Epoch 6/10 | Loss: 0.0029 | Val Acc: 100.00%
Epoch 7/10 | Loss: 0.1029 | Val Acc: 100.00%
Epoch 8/10 | Loss: 0.0673 | Val Acc: 100.00%
Epoch 9/10 | Loss: 0.0019 | Val Acc: 100.00%
Epoch 10/10 | Loss: 0.0045 | Val Acc: 100.00%
```